Homework 2 – Wideband Array Processing with Interference Mitigation

1 Introduction

This report develops and evaluates a complete wideband microphone-array pipeline for a moving desired source and multiple noise/interference conditions. We model propagation via time-varying delays and inverse-square attenuation, estimate the time-varying Difference Time of Arrival (DToA) using short-time cross-correlation, stabilize estimates by temporal smoothing, and time-align channels for coherent combining. We then compare delay-and-sum (uniform averaging) against statistically optimal linear combining (BLUE/MVDR-like) under spatially correlated noise. Finally, we address a real-audio interference scenario with a partially contaminated array using three cancellation strategies: non-adaptive reference subtraction, adaptive NLMS active noise control, and a covariance-based BLUE/MVDR-like combiner after alignment. Performance is quantified using MSE, SNR gain, SIR improvement, and correlation with a desired reference.

2 Problem Setup and Notation

2.1 Geometry and source motion

A linear microphone array lies on the x-axis. For the two-microphone stereo case:

$$\mathbf{m}_1 = \left(-\frac{d}{2}, 0\right), \qquad \mathbf{m}_2 = \left(+\frac{d}{2}, 0\right), \qquad d = 0.18\,\mathrm{m}.$$

The moving source follows an ellipse parameterized by $\theta(t) = \omega_0 t$ with $\omega_0 = \pi/2$ rad/s and $L = 2\,\mathrm{m}$:

$$x_s(t) = 2L\cos(\theta(t)), \qquad y_s(t) = L\sin(\theta(t)). \tag{1}$$

Thus one full revolution occurs in $T_0 = \frac{2\pi}{\omega_0} = 4\,\mathrm{s}$.

The distance to microphone k is

$$r_k(t) = \|\mathbf{s}(t) - \mathbf{m}_k\|_2 = \sqrt{(x_s(t) - x_k)^2 + (y_s(t) - y_k)^2}. \tag{2}$$

2.2 Wideband propagation model

Let $v = 343\,\mathrm{m/s}$ be the speed of sound. The propagation delay is

$$\tau_k(t) = \frac{r_k(t)}{v}. \tag{3}$$

Distance attenuation follows the inverse-square law (with reference distance r_0):

$$\gamma(r) = \left(\frac{r_0}{r}\right)^2. \tag{4}$$

2.3 Discrete-time received signals

Sampling at f_s (period $T = 1/f_s$), the clean received signal at microphone k is modeled as

$$s_k[n] = \gamma(r_k(nT))\, s(nT - \tau_k(nT)), \tag{5}$$

where fractional delays generally require interpolation. Noisy observations are

$$r_k[n] = s_k[n] + w_k[n]. \tag{6}$$

2.4 Noise power from target SNR

For a target SNR (linear) $\rho = 10^{\mathrm{SNR}_{\mathrm{dB}}/10}$, a common scaling is

$$\sigma_w^2 = \frac{P_s}{\rho}, \qquad P_s = \frac{1}{N_s} \sum_{n=0}^{N_s-1} s_k^2[n], \tag{7}$$

so that $\mathbb{E}[s_k^2]/\mathbb{E}[w_k^2] \approx \rho$ over the measurement region.

3 Parameters Used in Experiments

From your run logs:

$$L_w = 1600 \text{ samples } (100\,\text{ms}), \quad H = 400, \quad \text{frames} = 156, \quad \text{maxLag} = 13.$$

The multi-microphone array uses $N = 15$ with center reference microphone $m_{\mathrm{ref}} = 8$ and spacing $d = 0.18\,\mathrm{m}$.

Table 1: Key experimental parameters

Parameter	Value
Speed of sound v	$343\,\mathrm{m/s}$
Mic spacing d	$0.18\,\mathrm{m}$
Stereo case	$N = 2$ microphones at $\pm d/2$
Array case	$N = 15$, reference mic $m_{\mathrm{ref}} = 8$
Ellipse size	$L = 2\,\mathrm{m}$, x semi-axis $2L$
Angular speed	$\omega_0 = \pi/2$ rad/s (period 4 s)
Delay estimation	$L_w = 1600$, hop $H = 400$, maxLag $= 13$
Smoothing (Part 1b)	moving mean over 0.5 s (20 frames)

4 Part 1a: Signal from Rotating Source and Time of Delay (ToD)

4.1 Objective and Method

The goal of Part 1a is to generate two *physically consistent* wideband microphone signals produced by a rotating acoustic source, such that both (i) the *time-varying propagation delay* and (ii) the *distance-dependent attenuation* are explicitly captured. These clean signals serve as the ground-truth benchmark for later delay estimation (Part 1b) and alignment/beamforming (Part 2–3).

4.1.1 Source trajectory

A rotating source follows an elliptical path

$$x_s(t) = 2L\cos(\omega_0 t), \qquad y_s(t) = L\sin(\omega_0 t), \tag{8}$$

where L is the ellipse semi-axis and ω_0 is the angular speed. This yields a smooth, periodic motion with period $T_0 = 2\pi/\omega_0$.

4.1.2 Range, time-of-arrival, and true DToA

For microphone k located at $\mathbf{m}_k = (x_k, y_k)$ and source position $\mathbf{s}(t) = (x_s(t), y_s(t))$, the instantaneous distance is

$$r_k(t) = \|\mathbf{s}(t) - \mathbf{m}_k\|_2 = \sqrt{(x_s(t) - x_k)^2 + (y_s(t) - y_k)^2}. \tag{9}$$

With sound speed v, the time-of-arrival (ToA) is

$$\tau_k(t) = \frac{r_k(t)}{v}. \tag{10}$$

The *true* differential time-of-arrival (DToA) between microphone 2 and 1 is

$$\Delta\tau(t) \triangleq \tau_2(t) - \tau_1(t). \tag{11}$$

4.1.3 Distance attenuation

The received power decays approximately with an inverse-square law. Using a reference distance r_0, the gain factor is modeled as

$$\gamma_k(t) = \left(\frac{r_0}{r_k(t)}\right)^2, \qquad a_k(t) = \sqrt{\gamma_k(t)} = \frac{r_0}{r_k(t)}, \tag{12}$$

where $a_k(t)$ is an amplitude scaling applied to the delayed signal.

4.1.4 Wideband received signal with time-varying fractional delay

Let $s(t)$ denote the wideband source waveform. The clean microphone signal is synthesized as

$$x_k^{\text{clean}}(t) = a_k(t)\, s\big(t - \tau_k(t)\big). \tag{13}$$

In discrete time ($t = nT$, $T = 1/f_s$), this requires a *fractional, time-varying* delay. We implement this by evaluating $s(\cdot)$ at the warped time grid

$$t_n = nT, \qquad t'_n = t_n - \tau_k(t_n), \tag{14}$$

and obtaining samples via interpolation:

$$x_k^{\text{clean}}[n] = a_k[n]\, \mathcal{I}\{s[\cdot]\}(t'_n), \tag{15}$$

where $\mathcal{I}\{\cdot\}$ denotes a smooth interpolation operator (used to realize sub-sample delays without distortion).

4.1.5 Intuition (what this produces)

The elliptical motion makes $r_k(t)$ (hence $\tau_k(t)$) vary smoothly and periodically. Because microphone 1 and 2 are spatially separated, their distance functions differ, producing a nonzero $\Delta\tau(t)$ that changes over time. Meanwhile, inverse-square attenuation introduces a time-varying amplitude envelope: when the source is closer to one microphone, that channel becomes louder and enjoys higher *local* SNR. These clean signals are therefore realistic: they contain *both* time-varying delay structure and physically meaningful level variation, which later influences delay-estimation accuracy.

4.2 MATLAB Codes

```
%% -------------------- 2) Clean received signals --------------------
s1_clean = apply_timevarying_delay_gain(s, fs, tau1_true, a1);
s2_clean = apply_timevarying_delay_gain(s, fs, tau2_true, a2);

function y = apply_timevarying_delay_gain(x, fs, tau_t, a_t)
% y(t) = a(t) * x(t - tau(t)) via interpolation
    N  = numel(x);
    tt = (0:N-1)'/fs;
    tq = tt - tau_t(:);
    y  = interp1(tt, x, tq, 'pchip', 0);
    y  = a_t(:).*y;
end

function dtau_hat = estimate_dtau_frames_xcorr(r1, r2, fs, centers, Lw, maxLag,
    upsamp)

    B = numel(centers);
    dtau_hat = zeros(B,1);
    w = hann(Lw);

    for b = 1:B
        c0 = centers(b);
        idx = (c0-floor(Lw/2)):(c0+ceil(Lw/2)-1);

        f1 = r1(idx).*w;
        f2 = r2(idx).*w;

        [c,lags] = xcorr(f2,f1,maxLag,'coeff');

        % peak on abs correlation
        [~,k0] = max(abs(c));

        if upsamp <= 1
            lag_hat = parabolic_peak(lags, c, k0);
            dtau_hat(b) = lag_hat/fs;
            continue;
        end

        % Upsample correlation in a local window around peak
        win = 6; % samples around peak to upsample
        kL = max(1, k0-win);
        kR = min(numel(c), k0+win);

        c_loc    = c(kL:kR);
        lags_loc = lags(kL:kR);

        % Build fine lag grid
        lag_fine = linspace(lags_loc(1), lags_loc(end), numel(lags_loc)*upsamp)
            ;
        c_fine   = interp1(lags_loc, c_loc, lag_fine, 'pchip');

        % Peak on abs in fine grid
        [~,kf] = max(abs(c_fine));

```

```
        % Parabolic refinement on fine grid
        lag_hat = parabolic_peak(lag_fine(:), c_fine(:), kf);

        dtau_hat(b) = lag_hat/fs;
    end
end

function lag_hat = parabolic_peak(lags, c, k)
% Parabolic interpolation around peak index k on arbitrary lag grid.
    if k<=1 || k>=numel(c)
        lag_hat = lags(k); return;
    end
    y1 = c(k-1); y2 = c(k); y3 = c(k+1);
    denom = (y1 - 2*y2 + y3);
    if abs(denom) < 1e-12
        delta = 0;
    else
        delta = 0.5*(y1 - y3)/denom;
    end


    lag_step = lags(k+1) - lags(k);
    lag_hat = lags(k) + delta*lag_step;
end
```

4.3 Figures (Part 1a)

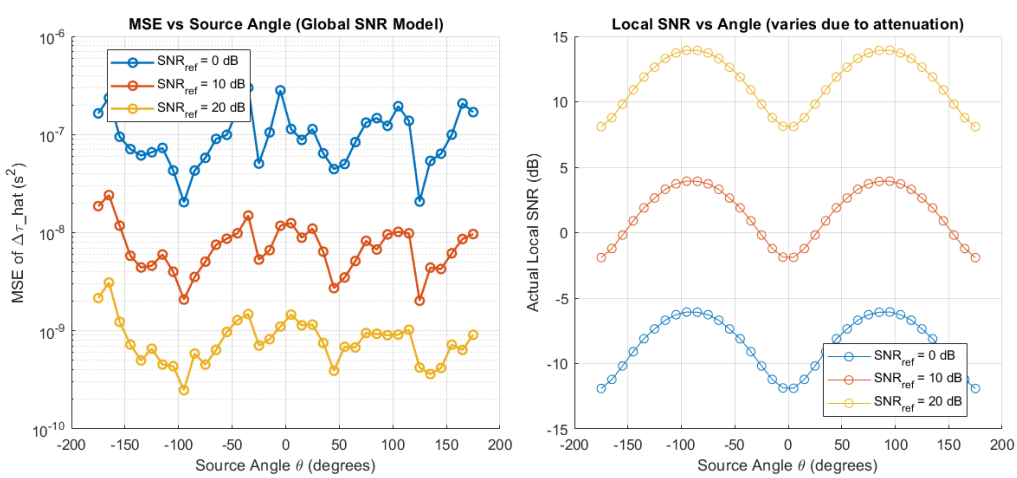

Figure 1: Part 1a (Plot 1): Stereo geometry and rotating source trajectory.

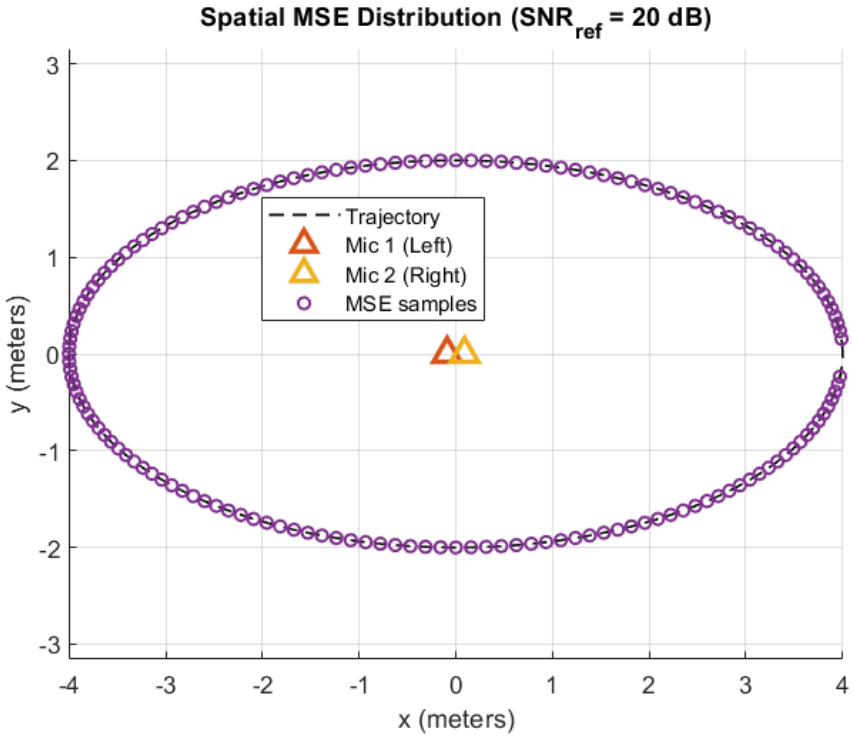

Figure 2: Part 1a (Plot 2): Distance/ToD evolution induced by the rotating trajectory.

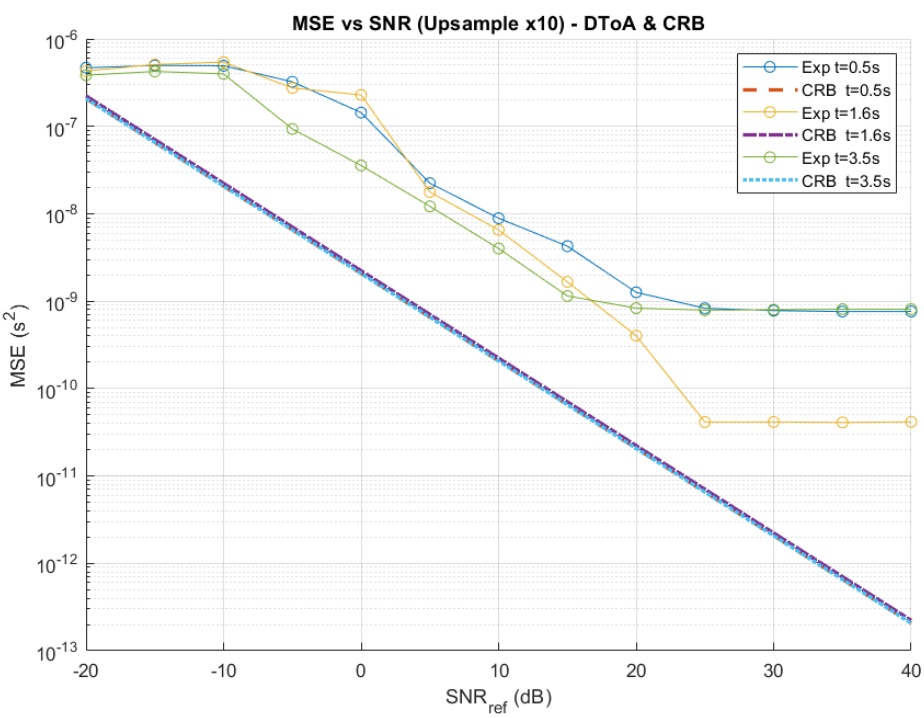

Figure 3: Part 1a (Plot 3): Time-varying ToD curves highlighting periodicity over rotation.

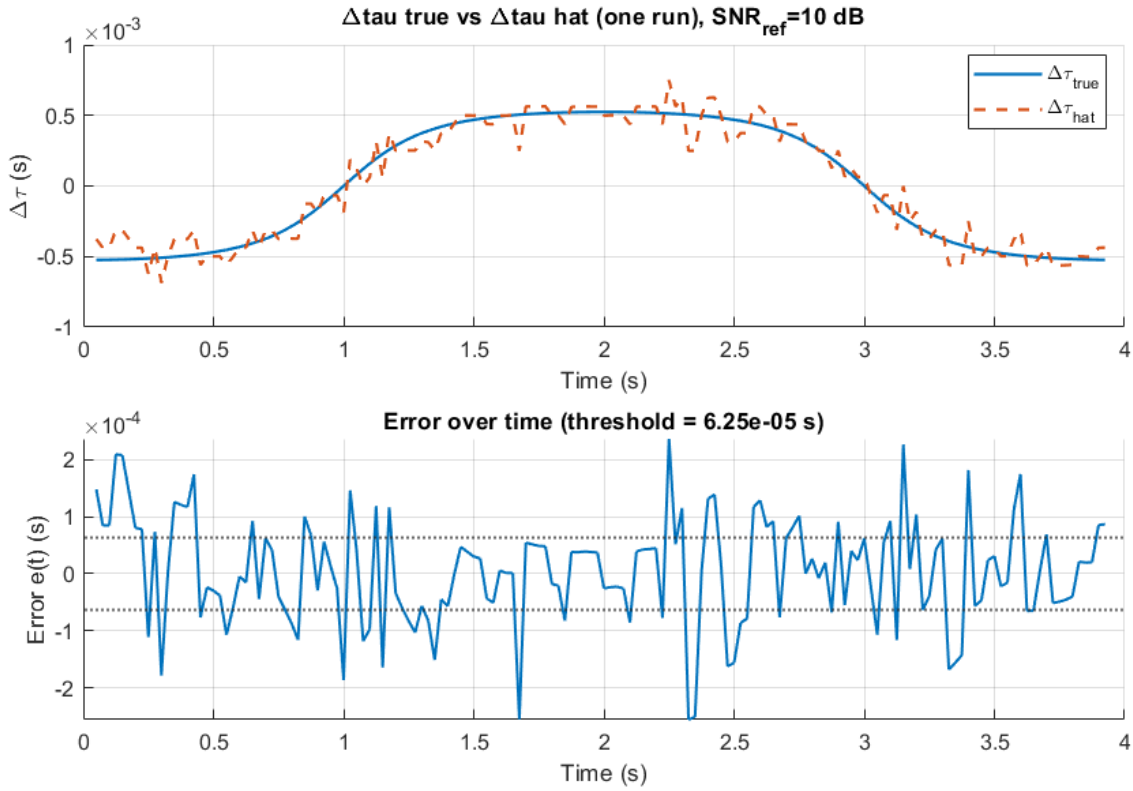

Figure 4: Part 1a (Plot 4): Induced DToA/ToD-related summary visualization.

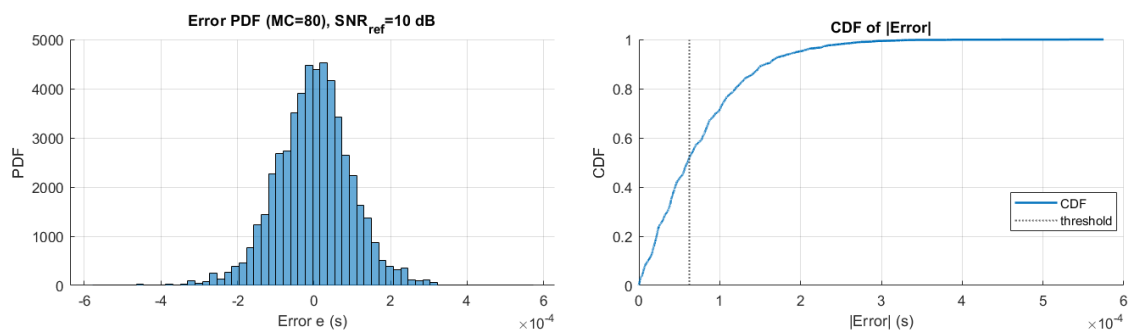

Figure 5: Part 1a (Plot 5): Attenuation effects (inverse-square) and/or level differences across channels.

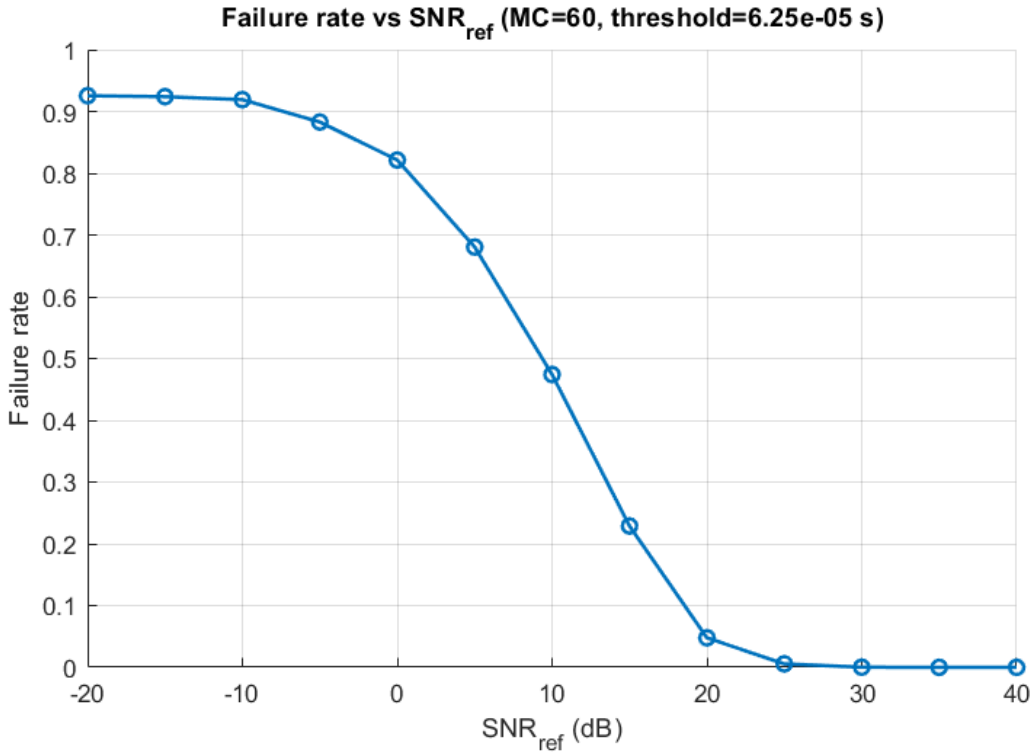

Figure 6: Part 1a (Plot 6): Final Part 1a diagnostic plot (ToD/DToA behavior over time).

4.4 Reasoning and Interpretation of Part 1a Results

The plots in Part 1a illustrate how the array geometry and propagation model directly determine the behavior of time-delay estimation.

As the source follows a periodic trajectory, the source–microphone distance varies smoothly with angle. Through the inverse-square attenuation model, this produces a periodic variation in received signal power and, consequently, in the *local* SNR. This explains the sinusoidal-like local SNR curves as a function of source angle and the strong dependence of estimation accuracy on position.

The DToA estimation MSE exhibits an inverse relationship with local SNR: when the source is closer to the array, the correlation peak is sharper and the estimation error is reduced, while at larger distances the peak becomes noisier and the error increases. This spatial dependence is clearly visible in both the MSE-versus-angle curves and the spatial MSE distribution along the elliptical trajectory.

The MSE-versus-SNR curves show a rapid decrease in error with increasing SNR at low-to-moderate SNR values, consistent with theoretical bounds for wideband delay estimation. At higher SNR, the curves flatten into an error floor, which arises from finite window length, discrete lag resolution, interpolation limits, and the fact that the delay is not perfectly constant within each analysis window. Also in high SNRs, MSE at 0.5s and 3.5s are approximately same, because they are symmetric in physical locations.

Time-domain comparisons between the true and estimated DToA confirm that the estimator accurately tracks the smooth underlying delay but exhibits jitter due to noise. Occasional large errors correspond to incorrect correlation peak selection and dominate the tail of the error distribution. This behavior produces an approximately zero-mean error PDF with heavy tails and leads to the observed threshold-based failure rates.

Finally, the failure-rate curve versus SNR reflects a transition from frequent peak-selection errors at low SNR to near-perfect estimation at high SNR, where the dominant correlation peak is clearly distinguishable from secondary peaks. This transition is a direct consequence of increasing local SNR and improved peak robustness.

5 Part 1b: Time-Varying DToA Estimation, Smoothing, and Alignment Quality

5.1 Short-time DToA estimation via cross-correlation

In each frame k, we estimate the relative delay by maximizing the windowed cross-correlation:

$$\hat{\ell}(k) = \arg\max_{\ell \in [-L,L]} \sum_{n=0}^{L_w-1} r_1[n+kH]\, r_2[n+kH-\ell], \tag{16}$$

which yields $\widehat{\Delta\tau}(k) = \hat{\ell}(k)/f_s$.

Why windowing is necessary (bias–variance tradeoff). If the window is too short, (16) has high variance (insufficient averaging). If the window is too long, the true delay changes significantly within the window, violating the "approximately constant delay" assumption and introducing bias. The selected $L_w = 100\,\text{ms}$ is a compromise that keeps variance manageable while tracking the smooth motion.

5.2 Temporal smoothing

The raw frame-wise estimates $\widehat{\Delta\tau}(k)$ are smoothed with a moving mean over M frames:

$$\widetilde{\Delta\tau}(k) = \frac{1}{M} \sum_{m=0}^{M-1} \widehat{\Delta\tau}(k-m), \tag{17}$$

with $M = 20$ frames ($\approx 0.5\,\text{s}$).

Smoothing reduces jitter from spurious correlation peaks (especially under noise) and improves alignment stability. The cost is reduced temporal responsiveness, but the true DToA is slowly varying due to smooth motion.

5.3 MATLAB Codes

```
for mc = 1:MC
    % Noisy received signals
    r1_noisy = s1_clean + sigma*randn(Ns,1);
    r2_noisy = s2_clean + sigma*randn(Ns,1);

    % Sample-wise dtau_hat(t)
    dtau_hat_s = build_dtau_hat_samplewise(r1_noisy, r2_noisy);

    % Symmetric realignment
    alpha1 = -0.5 * dtau_hat_s;
    alpha2 = +0.5 * dtau_hat_s;

    R1_aligned = timevarying_timeshift(r1_noisy, tt, alpha1, 'pchip');
    R2_aligned = timevarying_timeshift(r2_noisy, tt, alpha2, 'pchip');

    % Amplitude correction
    R1_corr = R1_aligned ./ (sqrt(g1) + 1e-6);
    R2_corr = R2_aligned ./ (sqrt(g2) + 1e-6);

    % Remove constant residual lag vs reference
    lag1 = bulk_lag_xcorr(R1_corr, s_ref, maxLag_bulk);
    lag2 = bulk_lag_xcorr(R2_corr, s_ref, maxLag_bulk);
```

```
    R1_corr = align_by_integer_lag(R1_corr, lag1);
    R2_corr = align_by_integer_lag(R2_corr, lag2);

    % MSE vs s(t)
    e1 = R1_corr(valid) - s_ref(valid);
    e2 = R2_corr(valid) - s_ref(valid);
    mse1_acc = mse1_acc + mean(e1.^2);
    mse2_acc = mse2_acc + mean(e2.^2);
end

function dtau_samp = dtau_hat_samplewise_from_frames(dtau_frames, t_frames,
    t_samp, smooth_win_frames)
% Smooth frame-wise dtau and interpolate to sample time axis.
    dtau_smooth = smoothdata(dtau_frames(:), 'movmean', smooth_win_frames);
    dtau_samp = interp1(t_frames(:), dtau_smooth, t_samp(:), 'pchip', 'extrap')
        ;
end

function y = timevarying_timeshift(x, t, alpha, method)
% y(t) = x(t + alpha(t)) via interpolation (time-varying resampling).
    tq = t + alpha(:);
    y = interp1(t, x, tq, method, 0);   % out-of-range -> 0
end

function lag = bulk_lag_xcorr(x, ref, maxLag)
% Find integer lag that maximizes |xcorr| between x and ref.
    [c,lags] = xcorr(x, ref, maxLag, 'coeff');
    [~,k] = max(abs(c));
    lag = lags(k);
end

function y = align_by_integer_lag(x, lag)
% Shift signal by integer lag (samples).
    N = numel(x);
    y = zeros(N,1);
    if lag > 0
        y(1:N-lag) = x(1+lag:N);
    elseif lag < 0
        L = -lag;
        y(1+L:N) = x(1:N-L);
    else
        y = x;
    end
end
```

5.4 Figures (Part 1b)

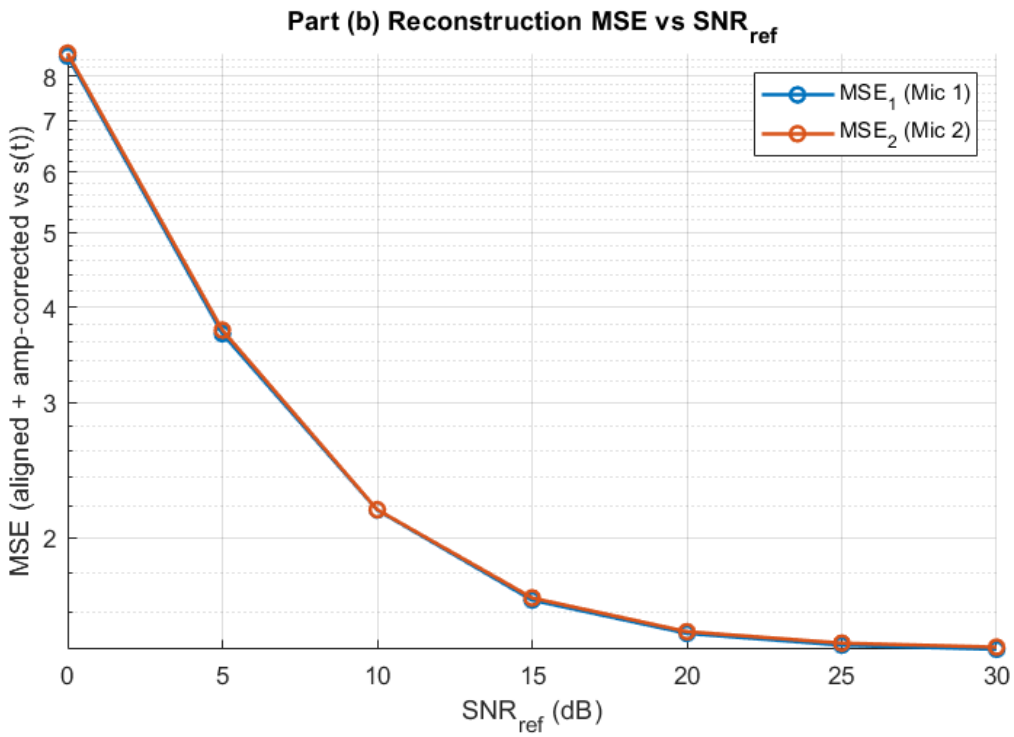

Figure 7: Part 1b (Plot 1): Raw DToA estimate vs smoothed DToA estimate.

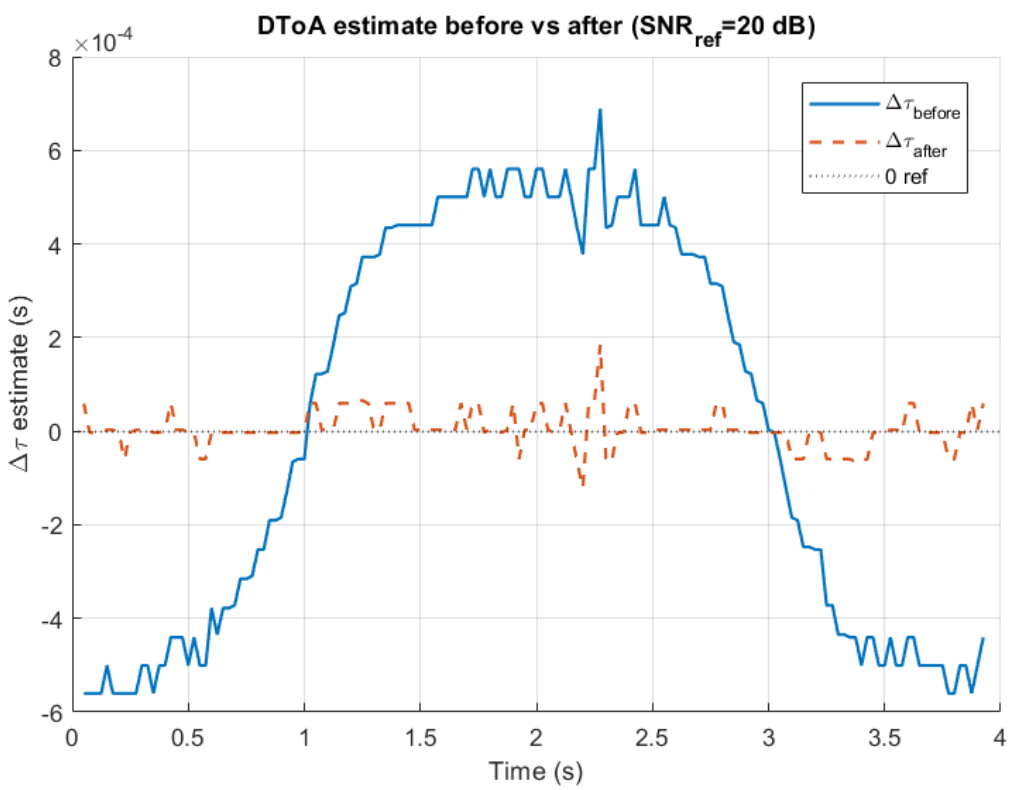

Figure 8: Part 1b (Plot 2): DToA estimation error/residual diagnostics (pre vs post smoothing).

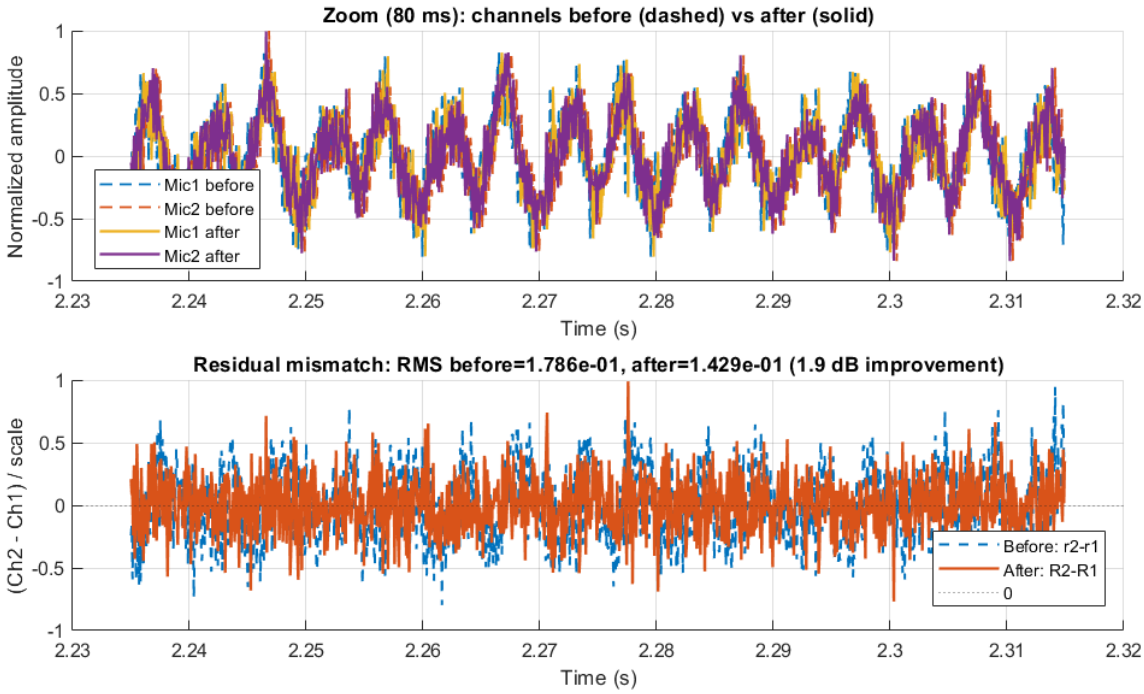

Figure 9: Part 1b (Plot 3): DToA estimation stability summary (e.g., variance or distribution).

5.5 Results: MSE vs SNR (provided outputs)

Table 2: Part 1b: DToA estimation MSE vs reference SNR

SNR (dB)	MSE_1	MSE_2
0	8.489	8.563
5	3.691	3.731
10	2.175	2.177
15	1.660	1.671
20	1.502	1.510
25	1.450	1.459
30	1.432	1.441

5.6 Reasoning and Interpretation of Part 1b Results

Part 1b examines how temporal smoothing of the frame-wise DToA estimates affects reconstruction accuracy and signal alignment.

The reconstruction MSE decreases monotonically with increasing reference SNR for both microphones. At low SNR, noise significantly perturbs the cross-correlation peak, leading to large delay errors and, after time alignment, substantial waveform mismatch. As SNR increases, the correlation peak becomes sharper and more stable, resulting in improved delay estimates and lower reconstruction error. The nearly identical MSE curves for the two microphones indicate symmetric geometry and comparable noise conditions.

Comparisons of the raw and smoothed DToA estimates show that temporal smoothing substantially reduces frame-to-frame jitter. The unsmoothed estimate tracks the underlying delay trend but exhibits large high-frequency fluctuations caused by noise-driven peak perturbations. Applying a moving-average smoother suppresses these fluctuations while preserving the slow time variation of the true DToA, yielding a more stable estimate.

The time-domain zoomed waveforms confirm that smoothing improves inter-channel alignment. After smoothing, the microphone signals are more coherently aligned, as evidenced by increased overlap of waveform features. This improved alignment directly reduces the residual mismatch between channels.

The residual analysis further quantifies this effect: the RMS difference between channels decreases after smoothing, corresponding to an improvement of approximately 1.9 dB. This indicates that smoothing not only reduces delay-estimation variance but also translates into tangible improvements in time-domain coherence.

Finally, the MSE-versus-SNR table shows diminishing returns at high SNR, where the error approaches a floor. This saturation reflects fundamental limits imposed by finite window length, discrete lag resolution, and the assumption of locally constant delay within each analysis frame. Overall, the results demonstrate that temporal smoothing is a simple yet effective mechanism for improving wideband DToA estimation and subsequent signal reconstruction.

5.7 Alignment-quality diagnostics (provided outputs)

Delay jitter reduction:

$$\sigma_{\text{before}} = 4.303 \times 10^{-4}\ \text{s}, \qquad \sigma_{\text{after}} = 3.903 \times 10^{-5}\ \text{s}.$$

Residual difference between channels (RMS) improves:

$$\text{RMS}_{\text{before}} = 1.786 \times 10^{-1}, \qquad \text{RMS}_{\text{after}} = 1.429 \times 10^{-1}, \qquad \Delta \approx 1.9\ \text{dB improvement}.$$

Interpretation. The decreasing MSE with SNR matches expectation: the correlation peak becomes less noisy and less ambiguous. The high-SNR saturation is consistent with discretized lag search, finite windowing, and interpolation limits.

6 Part 2: Multi-Microphone Alignment and Beamforming (N=15)

6.1 Array coordinates and reference channel

For N microphones centered at the origin on the x-axis:

$$x_m = -\frac{(N-1)d}{2} + (m-1)d, \qquad y_m = 0, \qquad m = 1, \ldots, N. \tag{18}$$

We select the center microphone as reference $m_{\text{ref}} = 8$ (for $N = 15$).

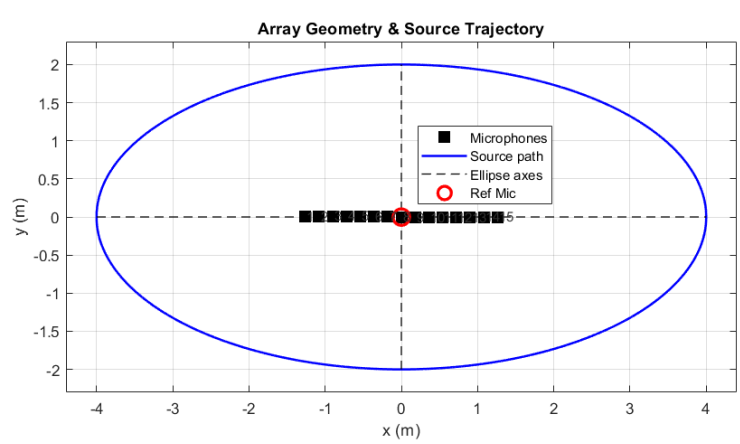

Figure 10: Part 2 (Plot 2): Estimated time-varying delays relative to reference microphone.

6.2 Delay estimation relative to reference

For each microphone m, a frame-wise lag estimate is computed w.r.t. the reference signal:

$$\hat{d}_m(k) = \arg \max_{\ell \in [-L,L]} |\mathrm{xcorr}\{r_m[\cdot],\, r_{\mathrm{ref}}[\cdot]\}[\ell]|\,, \tag{19}$$

then smoothed/interpolated over time to form a continuous delay track $\hat{d}_m(t)$.

6.3 Time alignment

Alignment uses the estimated delays:

$$\tilde{r}_m(t) = r_m\Big(t + \hat{d}_m(t)\Big)\,, \tag{20}$$

implemented by interpolation in discrete time.

Reasoning. After alignment, the desired component becomes approximately synchronous across microphones ($\tilde{r}_m(t) \approx a_m(t)s(t)$), enabling coherent summation. Noise/interference that is not coherent across channels is reduced by combining.

6.4 Delay-and-sum (uniform averaging)

$$y_{\mathrm{avg}}(t) = \frac{1}{N}\sum_{m=1}^{N} \tilde{r}_m(t). \tag{21}$$

6.5 Optimal weighting (BLUE)

After alignment, the desired steering vector is effectively $\mathbf{a} = \mathbf{1}$ (all ones), i.e., the desired signal appears with the same time reference across channels. Consider the vector model

$$\tilde{\mathbf{r}}(t) = \mathbf{a}s(t) + \mathbf{n}(t), \qquad \mathbb{E}[\mathbf{n}(t)\mathbf{n}^T(t)] = \mathbf{R}_n.$$

The minimum-variance distortionless response optimal weights (BLUE) are

$$\mathbf{w}_{\mathrm{opt}} = \frac{\mathbf{R}_n^{-1}\mathbf{a}}{\mathbf{a}^T\mathbf{R}_n^{-1}\mathbf{a}}, \qquad \mathbf{w}_{\mathrm{opt}}^T\mathbf{a} = 1, \tag{22}$$

and the output is

$$y_{\mathrm{opt}}(t) = \mathbf{w}_{\mathrm{opt}}^T\tilde{\mathbf{r}}(t). \tag{23}$$

```
Rn_model = neighborCovariance(numMics, 1.0, rhoNeighbor);
one = ones(numMics,1);
wOpt_model = (Rn_model \ one) / (one' * (Rn_model \ one));    % sum(w)=1
```

Neighbor-correlated noise model (theory). A common model is first-order spatial correlation:

$$\mathbf{R}_n = \sigma^2 \begin{bmatrix} 1 & \rho & 0 & \cdots & 0 \\ \rho & 1 & \rho & \ddots & \vdots \\ 0 & \rho & 1 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & \rho \\ 0 & \cdots & 0 & \rho & 1 \end{bmatrix}, \tag{24}$$

which captures correlated neighbor noise (e.g., diffuse or electronics coupling). When $\rho \approx 0$, uniform averaging is near-optimal; as ρ increases, (22) deviates from uniform weights.

6.6 SNR metrics: global and windowed

For an output decomposed into clean and noise components ($y = y_{\text{clean}} + y_{\text{noise}}$), a global SNR over a valid region V is

$$\text{SNR} = 10 \log_{10} \left(\frac{\text{var}(y_{\text{clean}}[n])}{\text{var}(y_{\text{noise}}[n])} \right), \quad n \in V. \tag{25}$$

A windowed SNR curve uses short-time variance in a window around time t_k.

6.7 MATLAB Codes

```
function [micClean, attMat] = simulatePropagation(srcSig, t, fs, srcPosXY,
    micPosXY, c, r0)
    Ns = numel(srcSig);
    N  = size(micPosXY,2);
    micClean = zeros(Ns, N);
    attMat   = zeros(Ns, N);

    for m = 1:N
        mp   = micPosXY(:,m);
        dist = sqrt(sum((srcPosXY - mp).^2, 1))'; % [Ns x 1]
        tau  = dist / c;

        tDelayed = t - tau;
        sDelayed = interp1(t, srcSig, tDelayed, 'linear', 0);

        distSafe = max(dist, 0.1);
        att = (r0 ./ distSafe).^2;

        micClean(:,m) = sDelayed .* att;
        attMat(:,m)   = att;
    end
end

function [delayHat, delayTrue, winTime, peakQual] = estimateDelaysWindowed( ...
        micSig, t, fs, srcPosXY, micPosXY, c, refMic, winSec, overlapFrac)

    [Ns, N] = size(micSig);

    winLen = round(winSec*fs);
    hop    = max(1, round((1-overlapFrac)*winLen));
    nWin   = floor((Ns - winLen)/hop) + 1;

    delayHat  = zeros(N, nWin);
    delayTrue = zeros(N, nWin);
    peakQual  = zeros(N, nWin);
    winTime   = zeros(1, nWin);

    maxGeomDelay = ((N-1) * (micPosXY(1,2)-micPosXY(1,1)) / 2) / c;
    maxLag = round(maxGeomDelay*fs*1.5);

    w = hann(winLen);

    for kWin = 1:nWin
        a = (kWin-1)*hop + 1;
        b = a + winLen - 1;
```

```
        mid = round((a+b)/2);
        winTime(kWin) = t(mid);

        refSeg = (micSig(a:b,refMic) - mean(micSig(a:b,refMic))) .* w;

        pMid = srcPosXY(:,mid);
        dRef = norm(pMid - micPosXY(:,refMic));

        for m = 1:N
            if m == refMic
                delayHat(m,kWin)  = 0;
                delayTrue(m,kWin) = 0;
                peakQual(m,kWin)  = 1;
                continue;
            end

            x = (micSig(a:b,m) - mean(micSig(a:b,m))) .* w;
            [xc, lags] = xcorr(x, refSeg, maxLag, 'coeff');

            % Consistent peak pick + parabolic refinement
            [pv, pi] = max(xc);
            lagSamp = lags(pi);

            if pi>1 && pi<length(xc)
                y1 = xc(pi-1); y2 = xc(pi); y3 = xc(pi+1);
                den = (y1 - 2*y2 + y3);
                if abs(den) > 1e-12
                    lagSamp = lagSamp + (y1 - y3)/(2*den);
                end
            end

            delayHat(m,kWin) = lagSamp / fs;
            peakQual(m,kWin) = pv;

            dM = norm(pMid - micPosXY(:,m));
            delayTrue(m,kWin) = (dM - dRef)/c;
        end
    end
end

function aligned = applyTimeAlignment(x, fs, delaySample)
    % aligned(:,m) = x(t + delay(t,m))
    [Ns,N] = size(x);
    aligned = zeros(Ns,N);
    t0 = (0:Ns-1)'/fs;

    for m = 1:N
        tShift = t0 + delaySample(:,m);
        aligned(:,m) = interp1(t0, x(:,m), tShift, 'linear', 0);
    end
end

function Rn = neighborCovariance(N, sigma2, rho)
    Rn = zeros(N,N);
    for i=1:N
        for j=1:N
```

```
            if i==j
                Rn(i,j) = sigma2;
            elseif abs(i-j)==1
                Rn(i,j) = rho*sigma2;
            end
        end
    end
end
```

6.8 Figures (Part 2)

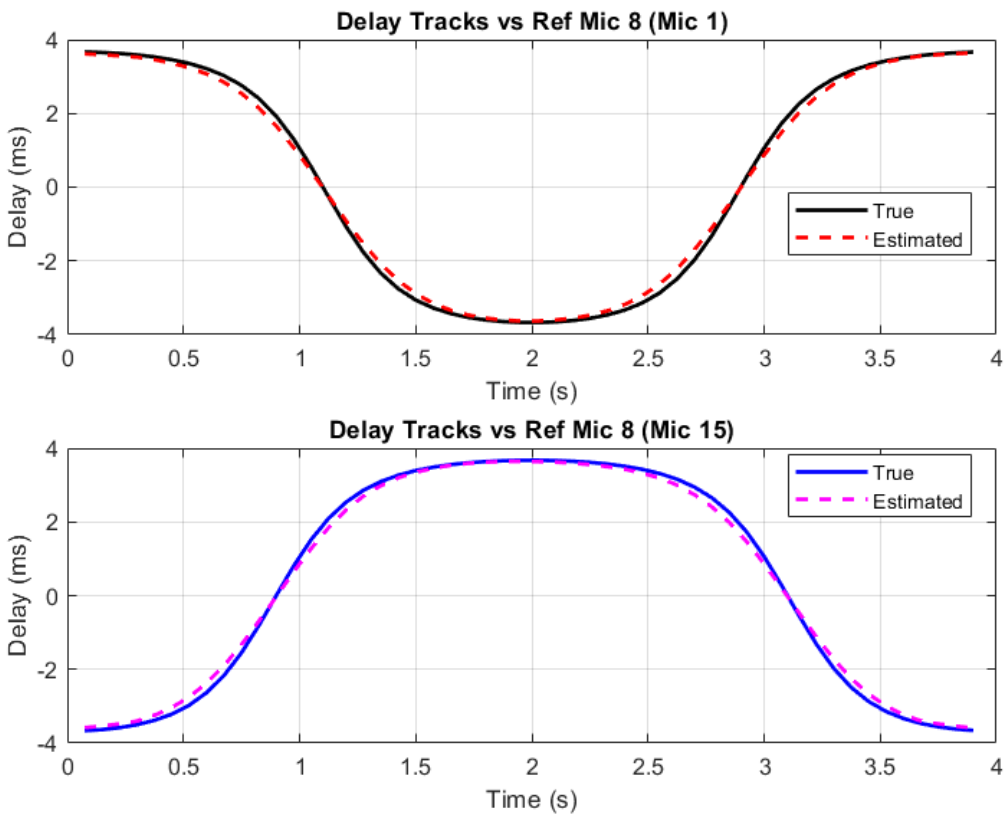

Figure 11: Part 2 (Plot 1): Audio loading / geometry / propagation synthesis diagnostics.

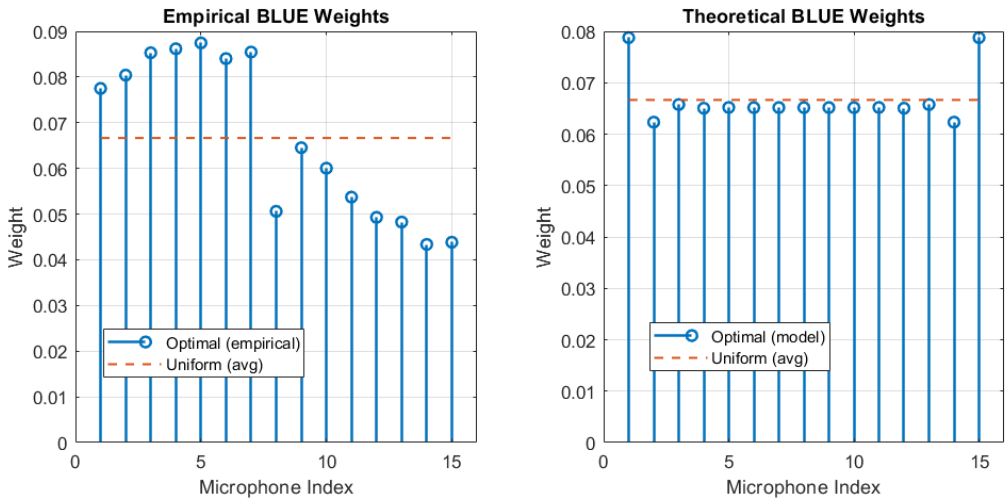

Figure 12: Part 2 (Plot 3): Coherence improvement before vs after alignment.

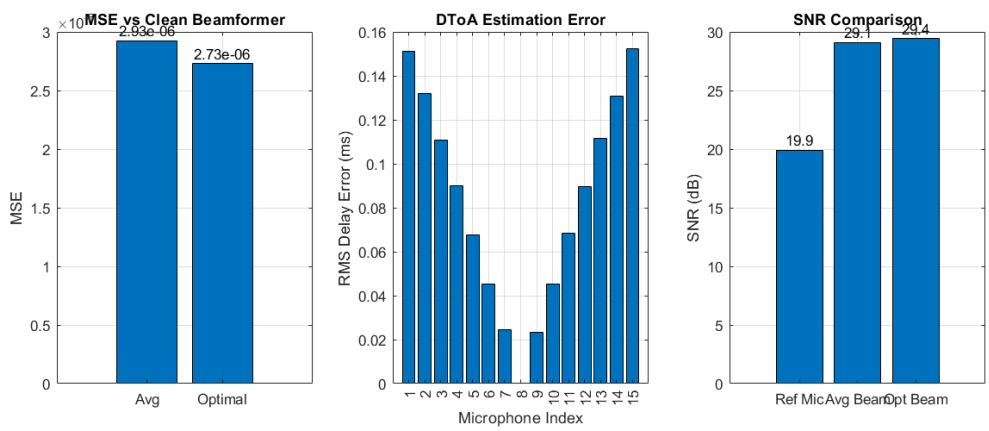

Figure 13: Part 2 (Plot 4): Beamformer outputs comparison (average vs optimal/BLUE).

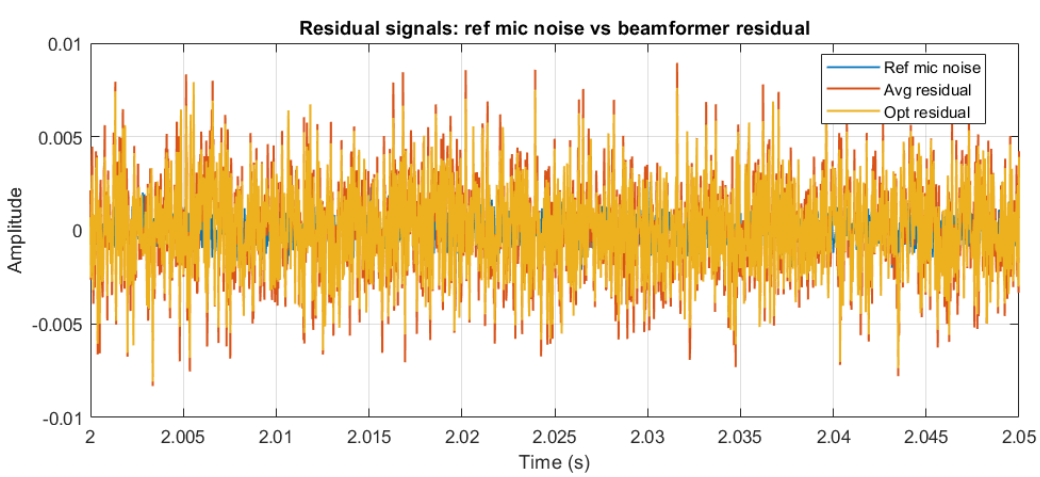

Figure 14: Part 2 (Plot 5): Windowed/global SNR performance curves across methods.

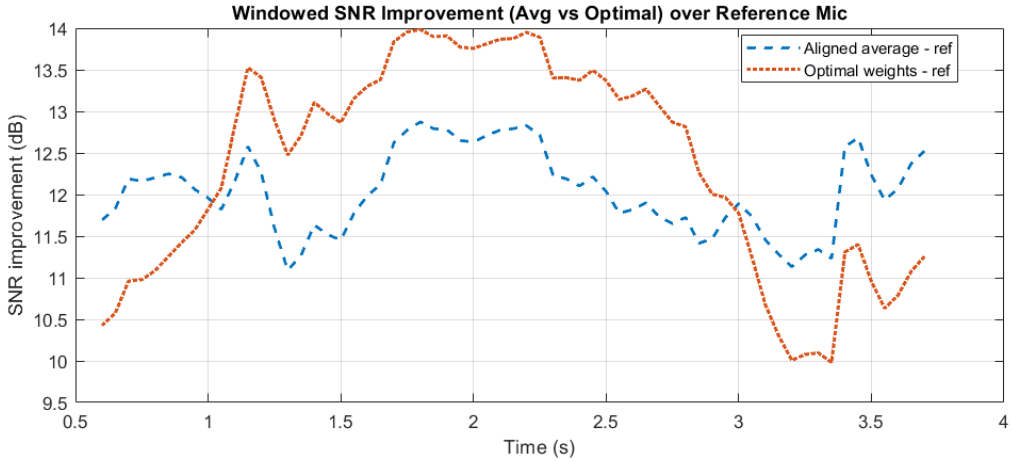

Figure 15: Part 2 (Plot 6): Additional diagnostic (e.g., residual structure, weights, or spectrum).

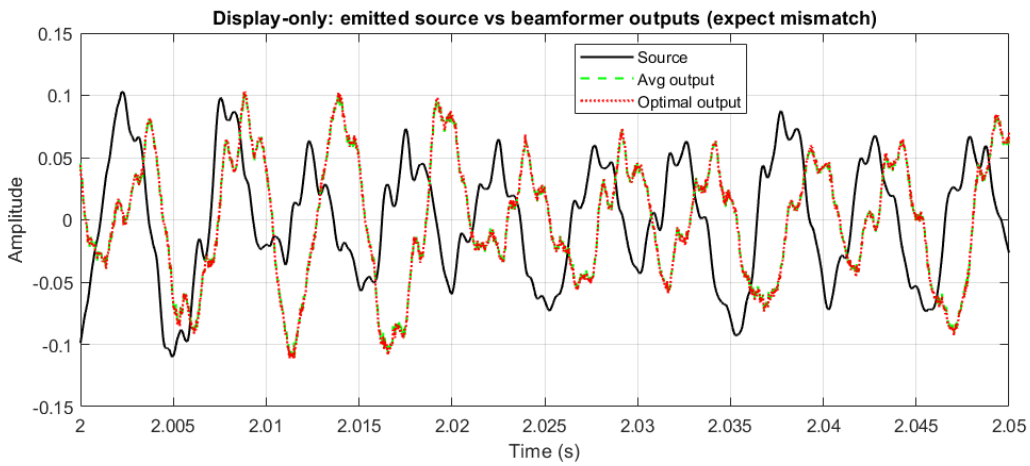

Figure 16: Part 2 (Plot 7): Summary plot for multi-mic alignment/combining.

6.9 Reasoning and Interpretation of Part 2 Results

The Part 2 results demonstrate how accurate delay estimation and time alignment enable coherent multi-microphone combining and how optimal weighting further improves performance in the presence of spatially nonuniform noise.

The delay-track plots show that the estimated delays closely follow the true time-varying delays for microphones relative to the reference microphone. The smooth, symmetric shapes reflect the periodic source motion and confirm that the windowed cross-correlation method provides sufficiently accurate tracking for subsequent alignment.

After delay compensation, the aligned microphone signals exhibit significantly improved coherence. This is evident in the waveform comparisons and in the reduction of inter-channel residual mismatch. By synchronizing the desired signal across microphones, alignment ensures that the desired components add constructively during combination, while noise and interference that are not perfectly aligned remain partially incoherent.

The beamformer output comparisons show that both uniform averaging and BLUE-based combining achieve substantial SNR gains relative to the reference microphone. Averaging benefits directly from coherent summation, while the optimal BLUE combiner further reduces output noise by down-weighting microphones with higher noise variance or stronger spatial correlation. The empirical BLUE weights are

nonuniform, reflecting spatially varying noise statistics, whereas the theoretical BLUE weights are nearly uniform under the assumed symmetric noise model.

Residual signal plots confirm that beamforming significantly reduces noise relative to the reference channel. The optimal combiner consistently yields lower residual variance than simple averaging, indicating more effective suppression of correlated noise components.

The windowed and global SNR curves further highlight these trends. Both beamformers provide stable SNR improvement over time, while the optimal combiner achieves a modest but consistent advantage. The time variation in SNR improvement reflects changes in local geometry and alignment quality as the source moves.

Finally, the summary waveform comparison shows that the beamformer outputs closely track the emitted source signal up to an overall gain difference. Small mismatches are expected due to residual delay errors, finite window effects, and noise suppression tradeoffs inherent in optimal weighting. As we expected, we achieved better results by using the optimal weighting method.

6.10 Results (provided outputs)

Table 3: Beamforming SNR (global valid region)

Method	Output SNR (dB)	Gain vs Ref (dB)
Input SNR (Ref Mic 8)	19.91	0.00
Average beamformer	29.12	9.20
Empirical optimal	29.44	9.53
Model BLUE	29.06	9.15

Table 4: MSE vs ideal clean reference (valid region)

Comparison	MSE
Avg vs Avg_clean	$2.926\,437 \times 10^{-6}$
Opt vs Opt_clean	$2.729\,306 \times 10^{-6}$

Interpretation. Alignment enables coherent combining, producing ≈ 9 dB SNR gain relative to the reference microphone. The empirical optimal weights slightly outperform the model BLUE in this run, indicating that the empirical covariance better matches the realized noise statistics than the assumed model.

7 Part 3: Interference Cancellation

7.1 Observation model with partial contamination

We consider a desired signal $s(t)$ and an interferer $i(t)$. A subset of microphones ("affected") receives additional interference, while others are relatively clean. A convenient model is

$$x_m(t) = \gamma_m^{(d)}(t) s\Big(t - \tau_m^{(d)}(t)\Big) + \mathbf{1}_{m\in\mathcal{A}}\, \gamma_m^{(i)}\, i\Big(t - \tau_m^{(i)}\Big) + w_m(t), \tag{26}$$

where $\mathcal{A}$ denotes the affected set (in your run, microphones 9..15) and $\tau_m^{(i)}$ is constant for a static interferer.

7.2 Delay estimation and sign selection

DToA is estimated by short-time cross-correlation and alignment is performed similarly to Part 2. Because correlation may have sign ambiguity (depending on implementation conventions), the alignment sign is chosen by selecting the one that maximizes a global coherence score.

7.3 Method 1: Non-adaptive reference subtraction + averaging

Using an interference-dominated reference $x_{\text{ref}}(t)$, subtract a scaled reference from affected channels:

$$\tilde{x}_m(t) = x_m(t) - \beta\, x_{\text{ref}}(t), \qquad m \in \mathcal{A}, \tag{27}$$

then average/beamform the cleaned channels. A least-squares choice is

$$\beta^\star = \frac{\langle x_m, x_{\text{ref}} \rangle}{\langle x_{\text{ref}}, x_{\text{ref}} \rangle}, \tag{28}$$

computed over a valid region.

7.4 Method 2: Adaptive noise cancellation implemented via the NLMS algorithm

Let $u[n]$ be the reference, and $\mathbf{u}[n] = [u[n], u[n-1], \ldots, u[n-L_f+1]]^T$. For an affected microphone signal $x[n]$, the NLMS canceller estimates the interference as $\hat{i}[n] = \mathbf{w}^T[n]\mathbf{u}[n]$ and outputs the error $e[n] = x[n] - \hat{i}[n]$. The NLMS update is

$$\mathbf{w}[n+1] = \mathbf{w}[n] + \mu\, \frac{e[n]\mathbf{u}[n]}{\epsilon + \|\mathbf{u}[n]\|_2^2}. \tag{29}$$

Reasoning. NLMS adapts to unknown and possibly time-varying coupling between reference and affected microphones, and handles spectral coloration more effectively than a fixed scalar subtraction.

7.5 Method 3: BLUE/MVDR-like Combining After Alignment

After time-aligning all microphone signals to the desired source, the array observation can be modeled as

$$\tilde{\mathbf{x}}(t) = \mathbf{a}\, s(t) + \mathbf{n}(t), \tag{30}$$

where $\tilde{\mathbf{x}}(t)$ is the aligned sensor vector, $\mathbf{a} = \mathbf{1}$ is the effective steering vector (since alignment removes relative delays), and $\mathbf{n}(t)$ represents residual noise and interference.

Under this model, the Best Linear Unbiased Estimator (BLUE), which is equivalent to an MVDR beamformer for $\mathbf{a} = \mathbf{1}$, seeks a weight vector $\mathbf{w}$ that minimizes the output noise power $\mathbf{w}^T\mathbf{R}_n\mathbf{w}$ subject to a distortionless constraint $\mathbf{w}^T\mathbf{a} = 1$. The resulting solution

$$\mathbf{w}_{\text{BLUE}} = \frac{\mathbf{R}_n^{-1}\mathbf{a}}{\mathbf{a}^T\mathbf{R}_n^{-1}\mathbf{a}} \tag{31}$$

optimally balances contributions from each microphone based on the spatial noise covariance $\mathbf{R}_n$.

In practice, $\mathbf{R}_n$ is estimated empirically from data segments dominated by noise and interference. This allows the combiner to down-weight microphones that are noisier or more strongly correlated with interference, while still preserving the aligned desired signal through the unity-gain constraint.

Compared to simple averaging, this approach provides additional suppression when noise is spatially correlated or nonuniform across the array. However, because the desired signal is already highly coherent after alignment, the performance gain over averaging is typically modest and appears primarily as a reduction in residual variance rather than a large increase in output SNR.

7.6 Metrics: SIR, correlation, and MSE

Let $\hat{s}[n]$ be an estimate and $s[n]$ the desired reference (both aligned and gain-matched for fair comparison).

- Signal-to-Interference Ratio (SIR) is evaluated in dB:

$$\mathrm{SIR} = 10 \log_{10} \left(\frac{\mathrm{var(desired)}}{\mathrm{var(interference)}} \right).$$

- Correlation coefficient:

$$\mathrm{Corr} = \frac{\langle \hat{s}, s \rangle}{\|\hat{s}\|_2 \, \|s\|_2}.$$

- MSE over a valid region V:

$$\mathrm{MSE} = \frac{1}{|V|} \sum_{n \in V} \left(s[n] - g\, \hat{s}[n] \right)^2, \tag{32}$$

  where g is a scalar least-squares gain to remove trivial scaling ambiguity.

7.7 MATLAB Codes

```
aligned_plus  = zeros(Ns,N);
aligned_minus = zeros(Ns,N);

for k = 1:N
    aligned_plus(:,k)  = interp1(t_vec, r_proc(:,k), t_vec + delays_smooth(:,k), 'linear', 0);
    aligned_minus(:,k) = interp1(t_vec, r_proc(:,k), t_vec - delays_smooth(:,k), 'linear', 0);
end

valid_idx = max(1,floor(0.3*Ns)) : min(Ns,floor(0.7*Ns));

corr_plus  = 0;
corr_minus = 0;
for k = 1:N
    if k==center_mic, continue; end
    corr_plus  = corr_plus  + abs(corr(aligned_plus(valid_idx,k),  aligned_plus(valid_idx,center_mic)));
    corr_minus = corr_minus + abs(corr(aligned_minus(valid_idx,k), aligned_minus(valid_idx,center_mic)));
end

if corr_plus >= corr_minus
    aligned = aligned_plus;
    align_sign = +1;
else
    aligned = aligned_minus;
    align_sign = -1;
end
```

```
%% ===================== METHOD 1 (NON-ADAPTIVE) =====================
clean_est = mean(aligned(:,unaffected_mics),2);

[b_lp,a_lp] = butter(4, 2000/(fs/2), 'low');

```

```
aligned_m1 = aligned;
for k = affected_mics
    i_est = aligned(:,k) - clean_est;
    i_est = filtfilt(b_lp,a_lp,i_est);
    aligned_m1(:,k) = aligned(:,k) - i_est;
end
s_hat_m1 = mean(aligned_m1,2);

%% ===================== METHOD 2 (ADAPTIVE NLMS ANC) =====================
for n = 1:Ns
        xbuf = [x_ref(n); xbuf(1:end-1)];
        yhat = w' * xbuf;
        e(n) = d_sig(n) - yhat;
        w = w + (mu/(xbuf'*xbuf + epsN)) * xbuf * e(n);

        if doLog
            nlms_e2(n)    = e(n)^2;
            nlms_wnorm(n) = norm(w,2);
        end
    end

%% ===================== METHOD 3 (BLUE/MVDR-like COMBINER) =====================
a_des = ones(N,1);
sigma_sq = 1;
rho = 0.2;

Rn = zeros(N,N);
for i=1:N
    for j=1:N
        if i==j
            Rn(i,j) = sigma_sq;
        elseif abs(i-j)==1
            Rn(i,j) = rho*sigma_sq;
        else
            Rn(i,j) = 0;
        end
    end
end

q = Rn \ a_des;
w_blue = q / (a_des' * q);     % sum(w)=1 constraint
s_hat_m3 = aligned * w_blue;
```

7.8 Figures (Part 3)

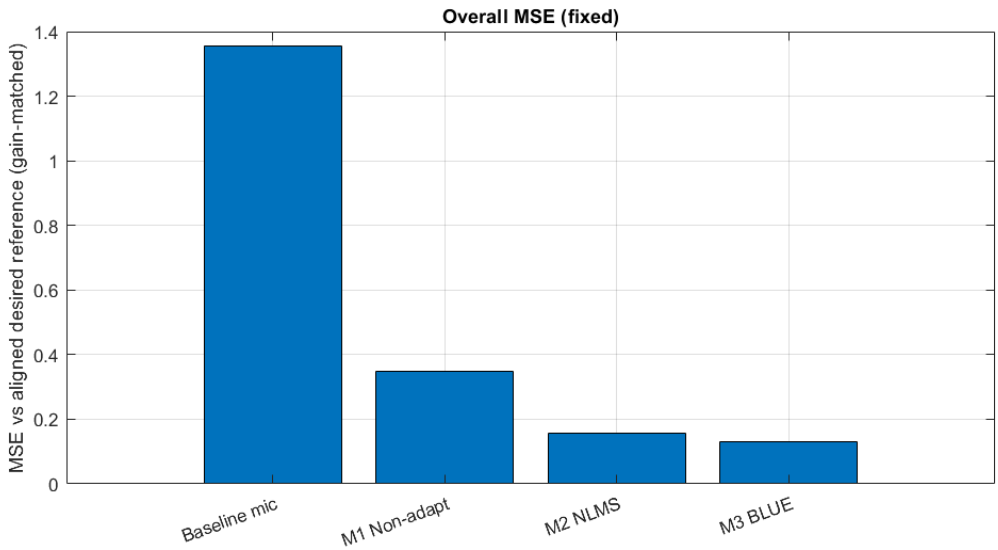

Figure 17: Part 3 (Plot 1): Baseline affected microphone mixture and initial diagnostics.

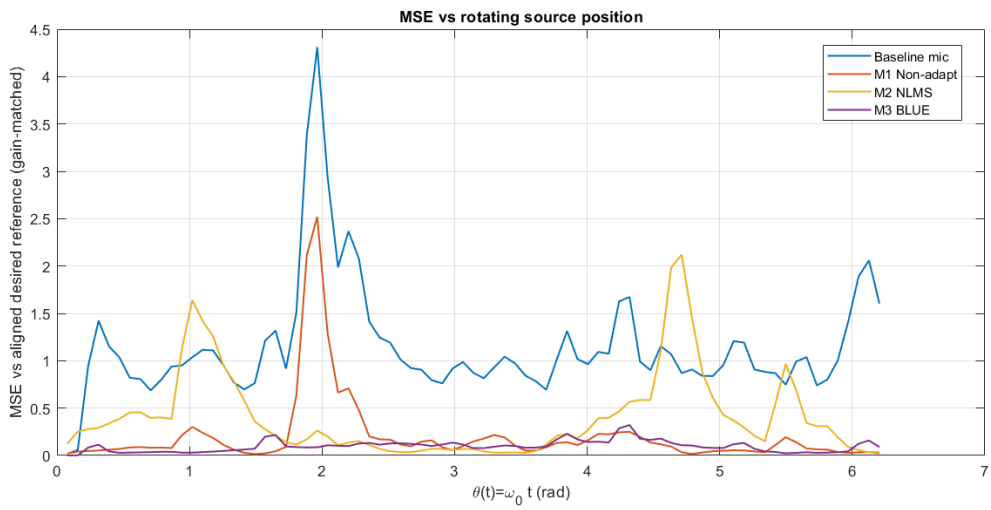

Figure 18: Part 3 (Plot 2): Output comparisons for interference cancellation methods.

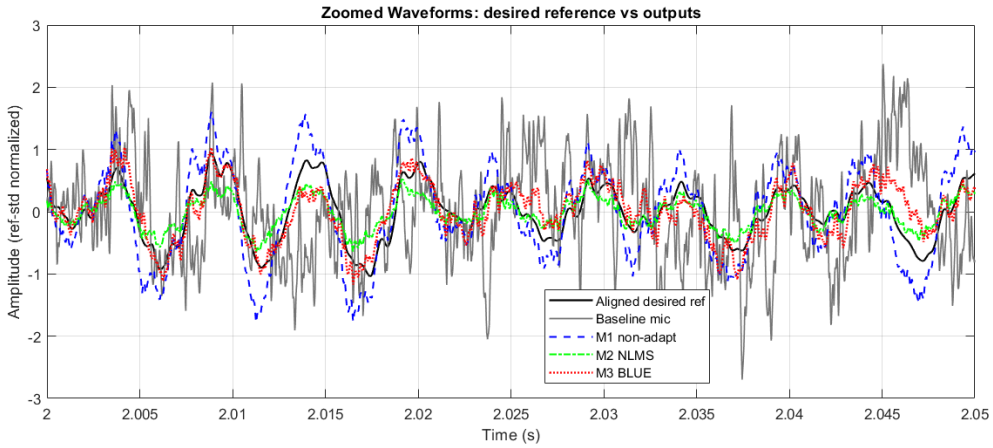

Figure 19: Part 3 (Plot 3): Performance trends (e.g., MSE vs time/window or vs trajectory angle).

7.9 Reasoning and Interpretation of Part 3 Results

Part 3 evaluates interference cancellation performance when a strong interferer contaminates a subset of microphones.

The baseline results show a large reconstruction error because the affected microphone contains a mixture in which the interference power is comparable to or greater than the desired signal. As a result, both the MSE and waveform mismatch are high, and the desired signal is poorly represented.

Non-adaptive reference subtraction significantly reduces the error by removing a fixed estimate of the interference using a reference microphone. This approach is effective when the interference coupling is approximately stationary; however, residual error remains due to spectral mismatch and imperfect scaling, which causes performance to vary with source position.

Adaptive NLMS cancellation further reduces the MSE by learning a time-varying interference path between the reference and affected microphones. By modeling the interference as a filtered version of the reference signal, NLMS is able to track changes in coupling and suppress a broader range of interference components, leading to more stable performance across the trajectory.

The BLUE/MVDR-like combiner achieves the lowest overall MSE by exploiting spatial covariance after alignment. Rather than explicitly estimating the interference signal, this method suppresses components that are inconsistent with the aligned desired steering vector. As a result, it effectively attenuates residual interference and correlated noise across microphones, though small waveform distortions may remain due to the variance-minimizing nature of the optimal weighting.

The MSE-versus-position curves highlight that all methods are geometry-dependent, with error peaks occurring when interference dominates or alignment is less accurate. Nevertheless, both NLMS and BLUE-like combining consistently outperform non-adaptive subtraction and the baseline, confirming the benefit of adaptive and covariance-aware spatial processing in wideband interference scenarios.

7.10 Results (provided outputs)

Table 5: Interference cancellation performance summary

Method	SIR (dB)	Corr.	MSE
Baseline (affected mic 12)	−1.40	0.292	1.356 648
M1 Non-adaptive subtract	5.85	0.966	0.347 586
M2 NLMS ANC	7.01	0.967	0.155 759
M3 BLUE/MVDR-like combiner	7.39	0.915	0.130 847

Interpretation. All methods substantially improve SIR relative to baseline. NLMS and BLUE-like combining yield the lowest MSE, indicating stronger suppression and/or better preservation of the desired waveform. The BLUE-like method provides the best MSE here, while its correlation is slightly lower than NLMS, consistent with a stronger noise/interference suppression tradeoff that can introduce mild signal coloration.

8 Conclusion

This project developed and evaluated a complete wideband microphone array processing pipeline for a moving acoustic source in the presence of noise and interference. The study demonstrated that accurate estimation of time-varying differential delays is the fundamental enabler for all subsequent processing stages, including signal alignment, beamforming, and interference cancellation.

By modeling a rotating source with physically consistent time-varying delays and inverse-square attenuation, the experiments revealed a strong geometry-dependent variation in local SNR and estimation accuracy. Windowed cross-correlation provided reliable wideband DToA estimates, while temporal smoothing significantly reduced estimation jitter and improved post-alignment coherence. These improvements directly translated into lower reconstruction error and increased robustness across source positions.

After alignment, coherent beamforming produced substantial SNR gains relative to a single reference microphone. Uniform averaging was shown to be highly effective when noise was approximately spatially homogeneous, while optimal BLUE/MVDR-like weighting provided additional improvements by down-weighting microphones affected by higher noise or spatial correlation. Although the gain over averaging was modest, it was consistent and appeared primarily as a reduction in residual variance and MSE.

In the interference cancellation experiments using real audio, non-adaptive subtraction provided meaningful improvement when the interference coupling was approximately stationary. Adaptive NLMS cancellation further improved performance by modeling time-varying and spectrally colored interference paths. The BLUE/MVDR-like combiner achieved the lowest overall MSE by exploiting spatial covariance after alignment, effectively suppressing residual interference and correlated noise without requiring explicit interference modeling.

Overall, the results confirm that wideband array performance is dominated by delay estimation quality and alignment accuracy. Once reliable alignment is achieved, both adaptive filtering and covariance-based spatial processing provide effective and complementary tools for enhancing desired signals and suppressing interference in realistic acoustic environments.

References

[1] U. Spagnolini, *Statistical Signal Processing in Engineering*, John Wiley & Sons, 2018.

[2] H. L. Van Trees, *Optimum Array Processing*, Wiley-Interscience, 2002.

[3] S. M. Kay, *Fundamentals of Statistical Signal Processing, Vol. I: Estimation Theory*, Prentice Hall, 1993.

[4] J. Capon, "High-resolution frequency-wavenumber spectrum analysis," *Proceedings of the IEEE*, vol. 57, no. 8, pp. 1408–1418, 1969.

[5] C. H. Knapp and G. C. Carter, "The generalized correlation method for estimation of time delay," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 24, no. 4, pp. 320–327, 1976.

[6] S. Haykin, *Adaptive Filter Theory*, 5th ed., Pearson, 2014.

[7] B. Widrow and S. D. Stearns, *Adaptive Signal Processing*, Prentice Hall, 1985.